

Automated Crack Detection in Concrete Surfaces Using Deep Transfer Learning

Gonzalo Morte Gómez
Faculty of Information and
Communication Technology
Wrocław University of Science
and Technology
Student ID: 293911

Arturo Hernández Martínez
Faculty of Information and
Communication Technology
Wrocław University of Science
and Technology
Student ID: 294664

Jose Daniel Moya Moreno
Faculty of Information and
Communication Technology
Wrocław University of Science
and Technology
Student ID: 293922

Abstract—This paper presents an automated computer vision system for detecting cracks in concrete surfaces, a critical task in structural health monitoring and quality control. Traditional manual inspections are subjective, time-consuming, and hazardous. At the same time, classical image processing methods often fail when dealing with real-world lighting and rough surface textures. To solve these limitations, we propose a deep learning pipeline based on a pre-trained ResNet50 architecture. The model was trained on the SDNET2018 dataset, which contains 56,092 labeled high-resolution images of concrete bridge decks, walls, and pavements. The dataset has a severe class imbalance (8,484 cracked vs. 47,608 non-cracked images, a ratio of roughly 1:5.6). We addressed this issue by using class weighting and real-time data augmentation. We used a two-stage training strategy: initial feature extraction with a frozen backbone, followed by full fine-tuning. At the standard decision threshold of 0.5, the model achieves 93 % overall accuracy with a recall of 0.78 for the cracked class. This is a massive improvement compared to the 0.48 recall of our MobileNetV2 baseline. By raising the threshold to 0.8, we shift the model to a configuration focused on safety. This adjustment increases the recall for the cracked class to 0.91 and reduces missed defects from 376 to 154. The results show that combining transfer learning, class weighting, and threshold adjustments provides a reliable and scalable solution for automated structural inspection.

Index Terms—computer vision, quality control, deep learning, transfer learning, ResNet50, crack detection, structural health monitoring, class imbalance.

I. INTRODUCTION

Structural health monitoring (SHM) plays a critical role in civil engineering. Early detection of surface defects is essential to ensure the durability, safety, and operational integrity of infrastructure such as bridges, pavements, and retaining walls. Surface cracks are primary indicators of underlying mechanical stress, material degradation, and environmental weathering. If left unaddressed, even minor cracks can rapidly turn into severe structural failures, increasing maintenance costs and jeopardizing public safety [1].

Historically, concrete infrastructure inspections have relied on manual visual assessments carried out by trained engineers, often in hazardous conditions requiring scaffolding or elevated platforms. Manual inspections are limited by subjectivity, human fatigue, high operational costs, and poor scalability. Automated computer vision systems solve these problems by

standardizing inspection criteria, enabling continuous monitoring, and integrating with hardware platforms like unmanned aerial vehicles (UAVs) and fixed cameras.

The objective of this project is to develop and evaluate an automated binary classification pipeline for concrete crack detection using deep Convolutional Neural Networks (CNNs). Using the SDNET2018 dataset [2], this study addresses the challenge of extreme class imbalance, which is a common issue in industrial anomaly detection. We also investigate the practical effects of adjusting the decision threshold after training. The primary contributions of this paper are:

- A direct comparison between a MobileNetV2 baseline and a fine-tuned ResNet50 architecture, showing a 30-percentage-point improvement in the recall of the cracked class.
- An analysis of how class weighting and real-time data augmentation work together to reduce the bias towards the majority class.
- A systematic evaluation of threshold tuning. We show that raising the decision threshold from 0.5 to 0.8 increases the recall of cracked surfaces from 0.78 to 0.91, reducing missed defects by 59 % without requiring any additional training.

The rest of this paper is organized as follows: Section II reviews relevant literature; Section III explains the dataset, preprocessing pipeline, and model architecture; Section IV presents the experimental results; and Section V concludes the paper and outlines future work.

II. RELATED WORK

Early attempts at automated crack detection mainly used classical image processing techniques. Methods such as Otsu's thresholding, Canny edge detection, and morphological operations were very popular due to their low computational cost [1]. While these methods work well under controlled laboratory conditions with perfect lighting, they fail in real-world scenarios. Concrete surfaces have rough textures, irregular shadows, stains, and debris that classical edge detectors often confuse with real cracks.

The focus shifted toward deep learning when large annotated datasets became available. Dorafshan *et al.* [1] provided a

rigorous comparison showing that CNNs significantly outperform classical filters on the SDNET2018 dataset. This dataset, introduced by Maguire *et al.* [2], created a standard benchmark covering bridge decks, walls, and pavements with crack widths ranging from 0.06 mm to 25 mm.

To balance computational cost and performance, transfer learning has become the standard strategy. Sandler *et al.* [3] introduced MobileNetV2, a lightweight architecture widely used as a baseline. On the other hand, He *et al.* [4] demonstrated that deeper networks like ResNet50 can extract much better features for complex texture patterns. A common challenge in all these studies is class imbalance. Since clean surfaces always outnumber defective ones, standard training tends to ignore the defects. Common solutions for this include using weighted loss functions, oversampling the minority class, and adjusting thresholds. We explore these strategies in this project.

III. METHODOLOGY

A. Dataset and Preprocessing

The SDNET2018 dataset [2] contains 56,092 labeled images of concrete surfaces across three structure types: bridge decks (D), walls (W), and pavements (P), each with cracked (C) and non-cracked (U) subdirectories. Fig. 1 illustrates a sample of these images, highlighting the variability in surface textures, lighting, and background noise (such as pitting and roughness) that complicate automated inspection.

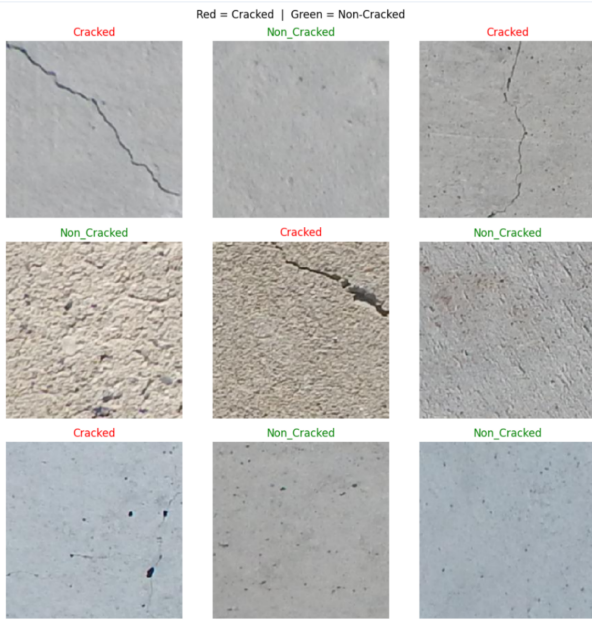


Fig. 1. Sample of *Cracked* and *Non_Cracked* concrete surface images from the SDNET2018 dataset, covering pavement, wall, and deck substrates. Images are resized to 224×224 pixels.

We reorganized the original folders into two target classes: *Cracked* and *Non_Cracked*. The resulting distribution contains 8,484 cracked and 47,608 non-cracked images, which means the non-cracked images are 5.6 times more frequent. We used

an 80/20 split with a fixed random seed (123) to produce 44,874 training images and 11,218 validation images. All images were resized to 224×224 pixels and normalized using the standard ResNet50 preprocessing function. The data pipeline was optimized with in-memory caching and prefetching to maximize the GPU speed.

B. Mitigating Class Imbalance

1) *Dynamic Class Weighting*: We modified the binary cross-entropy loss function to heavily penalize errors made in the minority class. We calculated the weights using this formula:

$$W_c = \frac{N_{\text{total}}}{2 \cdot N_c} \quad (1)$$

where W_c is the weight for class c , N_c is the number of samples in that class, and N_{total} is the total number of training images. This gives a weight of approximately 3.31 for the *Cracked* class and 0.59 for the *Non_Cracked* class. This means the model is punished over five times more for missing a crack than for triggering a false alarm.

2) *Real-Time Data Augmentation*: We added a data augmentation block directly into the model to simulate different real-world conditions. It applies random horizontal and vertical flips, random rotations (up to $\pm 20\%$), and random contrast adjustments ($\pm 20\%$) during the training phase.

C. Model Architecture

We selected the ResNet50 model, pre-trained on ImageNet, as our feature extractor. We replaced its original 1,000-class head with a custom classification head containing: (1) a Global Average Pooling (GAP) layer to reduce the $7 \times 7 \times 2048$ feature map to a 2048-dimensional vector; (2) a Dropout layer (rate = 0.2) to prevent overfitting; and (3) a single Dense neuron with a sigmoid activation function to output the crack probability.

D. Two-Stage Training Strategy

All experiments were run on an NVIDIA GeForce RTX 5060 GPU using TensorFlow 2.10 and the Adam optimizer.

Stage 1 (Feature Extraction for 10 epochs): The ResNet50 backbone was frozen, and only our custom classification head was trained at a learning rate of 10^{-3} . This adapts the head while keeping the ImageNet features intact.

Stage 2 (Full Fine-Tuning for 5 epochs): The entire network was unfrozen and retrained at a much lower learning rate of 10^{-5} . This allows the backbone to adapt specifically to concrete textures without forgetting the general shapes it learned from ImageNet. Class weights were applied in both stages.

E. Decision-Threshold Tuning

By default, the model classifies an image as *Cracked* if its probability is greater than 0.5. This default threshold maximizes overall accuracy, but in safety-critical inspections, a false negative (missing a real crack) is much more dangerous than a false positive (an unnecessary manual check). Because of this, we evaluated two operating points: a balanced threshold at 0.5 and a safer threshold at 0.8.

TABLE I
PERFORMANCE COMPARISON ON THE VALIDATION SET

Model / Threshold	Acc.	Prec.	Recall	F1
<i>Cracked class</i>				
MobileNetV2 (baseline)	0.91	0.88	0.48	0.62
ResNet50 ($\tau=0.5$)	0.93	0.78	0.78	0.78
ResNet50 ($\tau=0.8$)	0.80	0.43	0.91	0.58
<i>Non-Cracked class</i>				
MobileNetV2 (baseline)	0.91	0.91	0.99	0.95
ResNet50 ($\tau=0.5$)	0.93	0.96	0.96	0.96
ResNet50 ($\tau=0.8$)	0.80	0.98	0.78	0.87

IV. RESULTS AND DISCUSSION

A. Baseline vs. Final Model

Table I compares the performance of the MobileNetV2 baseline (frozen, 3 epochs, no class weights) against our final ResNet50 model at both threshold settings, evaluated on the 11,218 validation images.

The MobileNetV2 baseline achieved 91 % overall accuracy but only found 48 % of the actual cracks, producing 882 false negatives. This proves that looking only at overall accuracy is a mistake when datasets are imbalanced. Our ResNet50 model at the 0.5 threshold improves the recall for the cracked class to 0.78. This is a 30-percentage-point gain in defect sensitivity, which is a direct result of using a deeper architecture, applying class weights, and running full fine-tuning.

B. Confusion Matrix Analysis

Fig. 2 shows the confusion matrix for the 0.5 threshold. Out of 1,708 cracked validation images, the model correctly identifies 1,332 and misses 376. Fig. 3 shows what happens when we raise the threshold to 0.8: the number of false negatives drops from 376 to 154 (a **59 % reduction**). As expected, the number of false positives increases from 380 to 2,092. In a civil engineering context, this trade-off is completely justified: a false alarm just requires a quick manual check, while a missed crack is a major safety hazard.

C. Discussion

Three main conclusions emerge from these results. First, the architecture and the training strategy are key for detecting defects. Replacing MobileNetV2 with a fine-tuned ResNet50 and adding class weights improved the F1-score of the cracked class from 0.62 to 0.78. This confirms that deeper networks are better at recognizing subtle crack textures. Second, threshold tuning is a powerful and free tool. Raising the threshold from 0.5 to 0.8 increased recall from 0.78 to 0.91 without requiring the model to be trained again. Third, while our system is very accurate, it is a binary classifier: it detects if there is a crack in a 224×224 image patch, but it does not tell us exactly where the crack is. In the future, using pixel-level segmentation could provide even more information for engineers.

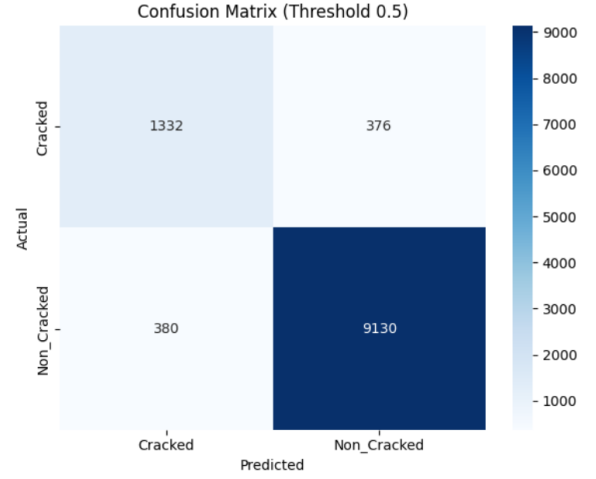


Fig. 2. Confusion matrix of the fine-tuned ResNet50 model at the standard threshold ($\tau = 0.5$). Overall accuracy is 93 %; recall for the cracked class is 0.78.

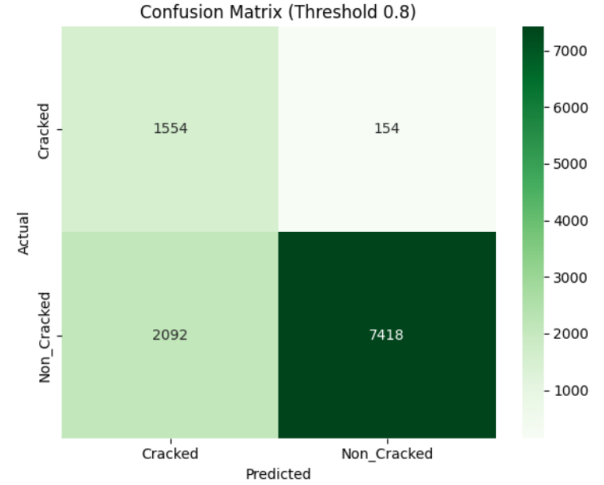


Fig. 3. Confusion matrix at the safety-focused threshold ($\tau = 0.8$). Recall for the cracked class rises to 0.91, reducing missed defects from 376 to 154.

V. CONCLUSION

In this project, we developed and evaluated an automated concrete crack detection pipeline based on ResNet50 transfer learning. By combining class weighting, real-time data augmentation, two-stage fine-tuning, and threshold adjustments, the system reaches a recall of 0.78 and 93 % overall accuracy under normal settings. When configured for maximum safety (0.8 threshold), the recall reaches 0.91, reducing missed defects by 59 % compared to the default setting. Considering that our initial baseline model missed more than half of the cracks, these results prove that our approach is highly reliable and ready for real-world structural inspections. Future improvements could focus on pixel-level crack segmentation and deploying the model on small hardware devices for real-time drone inspections.

ACKNOWLEDGMENT

We would like to thank the Faculty of Information and Communication Technology at Wrocław University of Science and Technology for providing academic guidance throughout this project.

REFERENCES

- [1] S. Dorafshan, R. J. Thomas, and M. Maguire, “Comparison of deep convolutional neural networks and edge detectors for image-based crack detection in concrete,” *Construction and Building Materials*, vol. 192, pp. 231–245, 2018.
- [2] M. Maguire, S. Dorafshan, and R. J. Thomas, “SDNET2018: An annotated image dataset for training, validation, and benchmarking of artificial intelligence-based concrete crack detection algorithms,” *Utah State University*, 2018.
- [3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 4510–4520.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778.